

最短路与最小生成树

请你们继续发布这样的聊天记录，这是我了解广州学生的唯一途径

Cap1taL

2026年7月

Floyd

- 可以在任何图上以 $O(n^3)$ 的时间复杂度, $O(n^2)$ 的空间复杂度求出任意两点间的最短路
- 关于 Floyd 的转移
- 朴素的 Floyd 为 $f_{k,x,y}$ 表示 $x \rightarrow y$ 间只经过 k 的点的最短路, 转移时考虑把 k 这个点插入即可。 $f_{k,x,y} = \min(f_{k-1,x,y}, f_{k-1,x,k} + f_{k-1,k,y})$
- 为什么可以省去第一维?
- 在用 f_{k-1} 更新 f_k 的过程中, 我们一直在使用 f 的第 k 行或第 k 列。而转移时, 实际上有 $f_{k,x,k} = \min(f_{k-1,x,k}, f_{k-1,x,k} + f_{k-1,k,k}) = f_{k-1,x,k}$, 即不会发生更新
- 因此直接省去第一维不会影响转移的正确性

Floyd的一些应用

- Floyd 本质上是在一个一个把点纳入考量
- 因此有很多可以“中继”的问题都可以用 Floyd 解决
- 例如传递闭包可以用 Floyd 做到 $O(\frac{n^3}{w})$
- 总之，不要只会背板子，而忘记了 Floyd 背后的思想

Bellman-Ford

- Bellman-Ford 可以在任何图上求出单源最短路，复杂度为 $O(nm)$
- 它的思想非常朴素：若 $dis_v > dis_u + w_{u,v}$ ，则松弛 v 的最短路为 $S \rightarrow \dots \rightarrow u \rightarrow v$
- 如何分析复杂度？显然，每次松弛都为一条路径延长了一条边，而最短路的长度不可能超过 $n - 1$ ，否则一定出现了环。因此最多进行 $n - 1$ 轮松弛操作
- 事实上，这也正是判断负环的方式：若 Bellman-Ford 第 n 轮还能成功松弛，则图中必定存在负环
- 需要注意的是，若 Bellman-Ford 正常结束，不能说明图中没有负环——有可能只是源点没有到访图中的负环。正确的做法是建立一个虚点，向每个点连一条边后从虚点跑 Bellman-Ford

SPFA



- 为什么要这样做？
- 在正权图上，我们永远应该使用 Dijkstra。
不卡 SPFA 是没有责任心的
- 在含负权图上/判负环时，原则上不应该
SPFA 可以通过而 Bellman-Ford 不能通过。
卡 Bellman-Ford 是没有道理的
- Bellman-Ford 更加简单好写

Dijkstra

- Dijkstra 可以在 $O(n^2 + m)$ 的时间内求出非负权图的单源最短路
- Dijkstra 的思想是，维护一个点集，每次取出点集中最短路最小的点，用来松弛出度点，并把出度点纳入点集
- 朴素实现是，暴力遍历点集内的所有点，寻找需要的最优点
- 可以用数据结构来优化这个过程

Dijkstra

使用 STL 的优先队列

- 直接用优先队列维护点集
- 松弛时，总是把点插入优先队列
- 取点时，使用一个 vis 数组，保证一个点只进行一次对外松弛
- 注意到松弛次数是 $O(m)$ 的，一个点会多次入队
- 复杂度为 $O(m \log m)$
- 在稠密图中表现不佳，甚至在 m 逼近 n^2 级别时不如朴素的 $O(n^2 + m)$ 实现

Dijkstra

使用支持 Decrease-Key 的堆

- 直接用堆维护点集
- 松弛时，如果点已经在点集内，则使用 Decrease-Key，否则直接插入点
- 使用斐波那契堆即可在 $O(n \log n + m)$ 的最优时间复杂度内完成 Dijkstra
- `__gnu_pbds` 内有斐波那契额堆 `thin_heap_tag` 可以做到上述复杂度，但常数很大，一般不使用

差分约束

- 有 $\{c_i\}$ ，且存在一系列形如 $c_i - c_j \leq k$ 的约束，可以整理为 $c_i \leq c_j + k$
- 我们发现它类似于松弛的式子，因此我们连接 $c_j \rightarrow c_i$ ，边权为 k ，跑一遍 Bellman-Ford 就可以判断负环（无解）或构造无解
- 一个技巧：若存在 $c_i = c_j$ 的限制，可以变成 $c_i - c_j \leq 0$ 和 $c_j - c_i \leq 0$ 。这个技巧在线性规划与对偶时也很常用

Johnson 全源最短路、同余最短路

- 我认为 NOIP 应该用不到

洛谷 P5471 [NOI2019] 弹跳

n 座城市在 $w \times h$ 网格上，每座城市有整数坐标。 m 个弹跳装置，每个位于某城市、花费 t_i 时间、可跳至给定矩形区域内的任意城市。求从首都（1 号城市）到每座其他城市的最短时间。

$$1 \leq n \leq 7 \times 10^4, 1 \leq m \leq 1.5 \times 10^5, 1 \leq w, h \leq n, 1 \leq t_i \leq 10^4$$

Solution

- 考察对 Dijkstra 的深刻理解……以及一点数据结构
- 直接暴力使用树套树优化建图，会被 128MB 的空间限制肘飞。尝试优化
- 考虑 Dijkstra 的流程，每次取出的点一定是当前确定的最短路最小点，而且一个点只会被取出拓展一次
- 这个题的建边方式首先保证了，每次我们取出的实际上是一个矩形
- 考虑当前矩形内的点，如果当前矩形内有点还属于其他先前取出过的矩形，那这个点对于当前矩形来说肯定是无用的
- 这启示我们不维护所有的边，而是每次取出一个最优矩形，找到矩形内的整点并扩展，最后再删除这些整点
- 显然每个点只会被删除拓展一次，使用优先队列，时间复杂度为 $O(m \log m + n \log^2 n)$ 。使用线段树套平衡树维护整点可以做到空间 $O(n \log n)$ ，具体实现可以使用 set。

洛谷 P4745 [CERC2017] Gambling Guide

加训期望……

n 个城市 m 条双向铁路。在 a 市投一枚硬币，随机得到一张到邻市的单程票。你可以选择立即使用或丢弃重买。求从 1 到 n 的最优策略下的期望花费。

$$1 \leq n, m \leq 3 \times 10^5$$

Solution

- 依旧考察对 Dijkstra 的理解
- 给出期望 DP 的式子。设 f_u 表示 $u \rightarrow n$ 的期望， d_u 表示度数

$$f_u = 1 + \frac{1}{d_u} \sum_v \min(f_u, f_v)$$

- 带着 min 是完蛋的，把它拆开。不妨设 $f_v < f_u$ 的 v 共有 k_u 个

$$f_u = 1 + \frac{1}{d_u} \sum_{v, f_v < f_u} f_v + \frac{(d_u - k_u)f_u}{d_u}$$

- 下面再做进一步整理

Solution

- 合并 f_u 得到

$$f_u = \frac{d_u}{k_u} + \sum_{v, f_v < f_u} \frac{f_v}{k_u}$$

- 这需要我们从小到大确定 f 并进行更新。因此使用 Dijkstra，从 f_n 开始执行
- 与 Dijkstra 的松弛不同，我们每次需要更新 k_v 后重新计算 f_v
- 作差可以证明 $f_v > f'_v > f_u$ ，因此这样更新是没有问题的

洛谷 P13271 [NOI2025] 机器人

我很喜欢这道题，不要问我为什么

有向图，每点的出边按 $1 \sim d_i$ 编号。机器人参数 p （初值 1，上限 k ），可在任意时刻花 v_p 加 1 或花 w_p 减 1。在路口 x 时可沿第 p 条出边移动。求从路口 1 到每个路口的最小总费用。

$$n, m \leq 3 \times 10^5, k \leq 2.5 \times 10^5$$

Solution

- 关键在于建模
- 类似动态规划地，容易想到根据状态拆点，把每个点拆成 k 个点，代表机器人此处的各个状态，然后根据状态转移连边
- 发现这样点数爆了，但又很容易想到大于 d_u 的点是无效的——它们只能暴力切换到 d_u
- 因此没有必要建出这些点，对于每个点只建立 d_u 个点，连边的时候如果 $p > d_v$ 就根据前缀和直接连到 d_v
- 复杂度是 Dijkstra 的复杂度

JOISC 2019 两种交通 / Two Transportations

N 座城市，Azer 知道 A 条铁路（边权 C_i ），Baijan 知道 B 条公交线路（边权 D_j ）。两人通过发送 0/1 字符通信（总数 ≤ 58000 ），协作求出从城市 0 到每座城市的最短路。

$N \leq 2000$, $A, B \leq 5 \times 10^5$, 边权 ≤ 500

Solution

- 稠密图，使用朴素 Dijkstra
- 每轮，双方把各自的最短路发过去，然后再让较小的那一侧把编号告诉另一个人，再维护最短路
- 直接发最短路长度会爆炸。此处用到了 Dijkstra 的性质，可以每次只发最短路的增量，注意边权 ≤ 500 ，因此需要 9 个比特。之后发送编号再需要 11 个比特，刚好 $2 \times 9 + 11 = 29 = \frac{58000}{2000}$

JOISC 2020 治療計画

N 个房屋排成一排，初始全部感染。有 M 个治疗方案 (T_i, L_i, R_i, C_i) ，在第 T_i 天晚上治愈 $[L_i, R_i]$ 。每天早上，每个感染者会感染相邻房屋。选择若干方案使所有村民全部被治愈，求最小花费，或输出 -1 。

$N, T_i, C_i \leq 10^9, M \leq 10^5$

Solution

- 尝试转化成最短路，我们需要一系列治疗来“封锁” $[1, n]$
- 治疗 x 可以“连边”到 y 当且仅当 $|t_x - t_y| \leq r_x - l_y$
- 连边时实际上是在保证不能从 y 的较小一侧偷偷传播感染者，而对较大一侧没有要求——因为我们最后取的是 $r = n$ 的点作为终点
- 分类讨论拆绝对值，有两个偏序关系，使用线段树优化建图，建立 t 上的线段树，每个节点维护一个 vector 暴力存储所有这个时间段内的所有点
- 类似刚才的弹跳一题，点权最短路一个点只会被松弛一次，因此松弛一次之后直接从线段树上删去即可，复杂度正确
- 时间复杂度为 $O(m \log m)$

Prim

- Prim 可以在 $O(n^2 + m)$ 的时间复杂度内，求出任意图的最小生成树
- 与 Dijkstra 类似，Prim 每次取出距离已有树集最近的点，进行更新拓展
- Prim 同样可以使用堆优化
 - 使用 STL 的优先队列可以做到 $O((n + m) \log n)$
 - 使用支持 $O(1)$ Decrease-Key 的堆（例如 `__gnu_pbds` 的斐波那契堆）可以做到 $O(n \log n + m)$
- 在稠密图上，朴素的 Prim 可能比 Kruskal 更快

Boruvka

- Boruvka 可以在 $O(m \log n)$ 的时间复杂度内求出最小生成树
- 初始把每个都视作做一个联通块，然后执行合并
- 每轮合并时，对于每个联通块求出与块外相连的最小边，并执行联通块合并
- 每轮至少联通块数减半，因此最多 $O(\log n)$ 轮
- 虽然看起来 Boruvka 的复杂度并不优越，但它的算法流程使得我们可以在一些具有特殊性质的图上使用它，尤其是图极度稠密而不能给出显式的边时

Kruskal

- 很常见的算法，在 $O(m \log m)$ 的时间复杂度内求出最小生成树
- 最边排序，然后贪心地选小边，用并查集维护连通性

Kruskal 重构树

- 在执行 Kruskal 的过程中，每发生一次合并，就建立一个新点，并把两个联通块根作为它的儿子，点权是边权
- 可以发现，所有叶子都是原图中的点
- 重构树上两个叶子的 LCA 的点权，对应原图中两点间路径最长边的最小值
- 同时，重构树上一个权值为 w 的点的子树内的所有叶子，对应原图中一个极大的、通过 $\leq w$ 边连接的联通块

洛谷 P4180 [BJWC2010] 严格次小生成树

无向图，求严格次小生成树的边权和（即边权和严格大于最小生成树的生成树中的最小值）。

$1 \leq N \leq 10^5$, $1 \leq M \leq 3 \times 10^5$, 边权 $\in [0, 10^9]$

Solution

- 首先求出最小生成树
- 枚举所有非树边，这会在 MST 上构成一个环
- 在环上找一个最长的树边删去
- 注意是严格次小，因此需要保证删去的边不与我们的非树边相等
- 因此需要在树上维护路径最大边和次大边
- 使用一些树上算法维护即可，例如倍增或者重链剖分

洛谷 P4768 [NOI2018] 归程

无向连通图，边有长度 l 和海拔 a 。每天给定出发点 v 和水位线 p ——所有海拔 $\leq p$ 的边有积水，不能开车但可步行。车可在任意节点弃用，之后步行回家（1 号节点）。求最小步行总长度，部分测试点强制在线。

$$n \leq 2 \times 10^5, m \leq 4 \times 10^5, Q \leq 4 \times 10^5$$

Solution

- 如今的归程已经成了一道模板题了
- 首先处理出每个点到 1 的最短路
- 拉出最大生成树的 Kruskal 重构树，可以开车访问的点就是一个子树
- 使用一些树上算法找到符合条件的子树，然后使用子树 min 即可得到答案

洛谷 P6765 [APIO2020] 交换城市

N 个城市 M 条双向路，边有油耗 $W[i]$ 。 Q 次询问，给定 X, Y ，两辆车分别从 X 出发去 Y 、从 Y 出发去 X ，要求两车在任何时刻不碰面（可以在任何城市等候任意时间，不在同城，不反向同时过同一条路）。求最小化两车行驶过道路的最大油耗。

$N \leq 10^5$, $M \leq 2 \times 10^5$, $Q \leq 2 \times 10^5$, $W[i] \leq 10^9$

Solution

- 类似瓶颈路
- 首先思考不碰面是在说什么——实际上就是要求瓶颈意义下，二者之间的联通块不是链
- 因此我们考虑改造一下 Kruskal 重构树
- 我们不止用并查集维护连通性，我们还维护联通块是不是链，以及如果是链的链端点
- 合并时如果链状态发生改变，我们也要建立新点
- 最终，在重构树上形如，每个子树依旧代表一个联通块，而下部的为链，上部的则不是链
- 每次询问找到 LCA 进行判断即可

AtCoder ARC098F Donation

无向连通图，每个点 v 有参数 (A_v, B_v) 。从任意点出发，到达点 v 时身上须有至少 A_v 元，此后可向其捐献 B_v 元（保证剩余 ≥ 0 ，或者暂时不捐献）。需对所有点各捐献一次，求初始最少携带钱数。

$$1 \leq N \leq 10^5, M \leq 10^5, A_i, B_i \leq 10^9$$

Solution

- 重构树神题，没有重构树就做吧一做一个不吱声
- 首先注意到，既然可以多次走过一个点，那么在最后一次经过时再捐钱更好
- 有了这个性质，我们考虑倒着做：每次到一个点需要你身上有 $a_u - b_u$ 的门槛钱，然后你可以在这里抢到 b_u 的钱，只能第一次来的时候抢
- 我们发现自己财源滚滚，所以抢过钱的点都可以随便走
- 这显然为我们提供了一个做法：枚举终点和剩余钱数，然后不断抢旁边门槛最低点的钱，拓展联通块。但这个做法复杂度太高注定没有前途，尝试优化

Solution

尝试加速抢钱

- 我们发现这个过程非常类似于瓶颈路的过程，即我们总是先访问 $\leq k$ 的块
- 给边赋权 $\max(a_u - b_u, a_v - b_v)$ ，然后拉出 Kruskal 重构树。
- 那么在重构树上，我们就要不断跳父亲，每跳一次就抢光整个子树里的所有点。考虑加速这个过程
- 不妨设 s_u 表示重构树上 u 子树所有的 b_i 之和，我们要求时刻 $r + s_u \geq a_{fa} - b_{fa}$ 成立。 r 表示终点剩余钱数
- 那么 r 显然 r 最小值是 $a_{fa} - b_{fa} - s_u$ 的最大值
- 在重构树上做一次 DFS，加点数据结构用脚维护

AtCoder cf17_final J Tree MST

树有 N 个顶点，边权 C_i ，点权 X_i 。定义 $f(u, v) = \text{dist}(u, v) + X_u + X_v$ 。考虑以 $f(u, v)$ 为边权的完全图 G ，求 G 的最小生成树总代价。
 $2 \leq N \leq 2 \times 10^5$, $X_i, C_i \leq 10^9$

Solution

- Boruvka 宣传片
- 一共会做 $O(\log)$ 轮
- 我们只需要解决，每轮怎么给每个联通块找到最小出边
- 若不考虑联通块的限制，单纯最小化 f , $dist$ 使得它可以简单树形 DP 解决
- 考虑联通块的限制也很简单——只需要维护最小值和次小值，并且钦定二者属于不同联通块即可转移
- 复杂度为 $O(n \log n)$

Codeforces 1648E Air Reform

原图 G 有 n 点 m 边，边权 c_i ，路径费用定义为路径上最大边权。补图 G' 包含所有 G 中没有的边，边权为 G 中两点间最小路径费用。对 G 中每条边，求 G' 中该两点间的最小路径费用。

$$n, m \leq 2 \times 10^5, \sum n, \sum m \leq 2 \times 10^5$$

Solution

- Boruvka 宣传片
- 首先拉出来原图的 Kruskal 重构树，两点在原图的最小路径费用就是 LCA 的点权
- 现在对补图用 Boruvka。尝试对每个点求出不同联通块的最小出边，我们发现两点的 LCA 越深，路径费用越小，体现在重构树上即为 dfn 越接近 LCA 越靠下
- 因此，我们只需要对每个点，找到 dfn 意义下左侧和右侧第一个“不同联通块”且“原图中无连边”的点即可
- 具体实现可以用 set 维护联通块，直接在联通块内跳 dfn，若有连边就再暴力跳。注意到因为连边而失败的尝试次数最多 m 次
- 复杂度为 $O((n + m) \log^2 n)$ ，使用链表替换 set 可以做到单 log，可能多了一些细节

Thanks!